ETSI TR 104 287 V1.1.1 (2026-08)



TECHNICAL REPORT

Methods for Testing and Specification (MTS);
Security Testing;
Security Validation Methodology for IoT components

Reference
DTR/MTS-TST12SecValMet

Keywords
IoT, methodology, ML, security, testing

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed, this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 4
Foreword 4
Modal verbs terminology 4
Executive summary 4
Introduction 5
1 Scope 7
2 References 7
2.1 Normative references 7
2.2 Informative references 7
3 Definition of terms, symbols and abbreviations 9
3.1 Terms 9
3.2 Symbols 9
3.3 Abbreviations 9
4 Security testing overview 10
4.1 General 10
4.2 Current state of SAST 11
4.2.0 General 11
4.2.1 Secure coding standards 11
4.2.2 Code review 12
4.2.3 Symbolic execution 12
4.2.4 Taint analysis 13
4.2.5 AI-driven static analysis 13
4.3 Current state of DAST 13
5 Specification of a security validation methodology 16
5.1 General 16
5.2 Static Application Security Testing (SAST) 17
5.3 Dynamic Application Security Testing (DAST) 18
5.4 Interactive Application Security Testing (IAST) 19
5.4.1 General 19
5.4.2 Verification of SAST findings 19
5.5 Firmware Fuzzing 22
5.5.0 Firmware Fuzzing Introduction 22
5.5.1 Static Memory containing global variables 22
5.5.2 Stack and Heap sanitization 23
6 Validation of security patches 23
6.1 Introduction 23
6.2 Genetic grey-box fuzzing for security patch validation 23
History 25

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Technical Report (TR) has been produced by ETSI Technical Committee Methods for Testing and Specification (MTS).

The aim of the present document is to outline a unified methodology for security validation of IoT components, with a particular focus on automated security testing techniques and their integration into supply-chain-oriented frameworks such as Device Security Passports (DSPs). The present document has been influenced by research and implementation results from the DOSS project ("Secure-by-Design IoT Operation with Supply Chain Control").

Modal verbs terminology

In the present document "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

The security of Internet of Things (IoT) systems depends critically on the trustworthiness of their constituent components, including firmware, embedded software, third-party libraries (incl. open-source libraries) and in-house applications. The present document describes a security validation methodology for IoT components that supports secure-by-design development and supply-chain-aware validation.

The methodology integrates:

- **Static Application Security Testing (SAST):** Early-lifecycle analysis of source and binary code to identify security weaknesses before deployment. The methodology combines rule-based analysis, secure-coding standards and AI-enabled vulnerability prediction and classification.
- **Dynamic Application Security Testing (DAST):** Runtime testing of binaries, firmware and services without requiring source code, with emphasis on coverage-guided fuzzing and specialized embedded-firmware fuzzing.
- **Interactive Application Security Testing (IAST):** Combined use of SAST and DAST results to focus dynamic testing on the most relevant code regions and to confirm or negate static analysis findings, thereby reducing false positives.
- **Security patch validation:** Test techniques to validate that a security patch fully addresses a vulnerability and does not leave alternative exploit paths open.

The methodology is designed to integrate with supply-chain-oriented artefacts such as a **Device Security Passport (DSP)** [i.7]. Testing artefacts (findings, coverage metrics, evidence of patch validation) are exposed to the DSP so that component manufacturers, integrators and operators can:

- assess component security posture;
- prioritize remediation activities;
- support regulatory and certification processes (for example in the context of the Cyber Resilience Act); and
- maintain an auditable security history across the full component life-cycle.

Introduction

The security of IoT systems is increasingly recognized as a critical concern across industrial, consumer and critical infrastructure domains. IoT products are typically assembled from heterogeneous hardware and software components originating from multiple vendors, open-source communities and in-house development efforts. These components are updated independently, reused across product families and operated in diverse environments for extended periods of time. This supply-chain-centric reality complicates traditional, monolithic approaches to security assurance and creates a need for methodologies that are capable of addressing components both individually and in the context of the wider system in which they operate.

At the same time, the regulatory landscape is evolving significantly. The European Cyber Resilience Act (CRA) [i.1] introduces binding obligations for the security of products with digital elements, including IoT devices, across their full lifecycle. These obligations require manufacturers, importers and distributors to demonstrate that their products have been designed, developed and maintained in accordance with appropriate security practices. Structured, repeatable and traceable security testing is therefore no longer a purely technical concern but is becoming a prerequisite for regulatory compliance and market access.

Existing security testing practices are, however, fragmented. The following limitations are observed in current industrial practice:

- Static analysis is often used in software development, but it often produces incorrect results and is not linked to dynamic testing. Dynamic testing and fuzzing campaigns are often scoped to individual products or selected interfaces, executed late in the development lifecycle.
- Lack of interactive techniques that combine static and dynamic analysis.
- Patch validation is commonly performed in an ad-hoc manner, with limited automation and insufficient traceability of the evidence generated.

These limitations are particularly consequential in IoT deployments, where the attack surface is distributed across many heterogeneous devices, communication interfaces, network services and backend components, and where the exploitation of a single residual vulnerability can compromise entire fleets of products simultaneously. The growing emphasis on regulatory frameworks such as the CRA further increases the need for reliable, auditable and repeatable security testing processes.

The DOSS project ("Secure-by-Design IoT Operation with Supply Chain Control") has addressed these challenges by introducing the Supply Trust Chain (STC) concept. The STC proposes a Device Security Passport (DSP) for components, which records security-relevant information across a component's full lifecycle, and a Component Tester (CT), which generates and provides automated security testing evidence to the DSP platform. The security testing methodology developed within DOSS integrates static, dynamic and interactive application security testing as well as structured patch validation into a coherent, supply-chain-aware process.

The present document generalizes and abstracts the methodology developed in the context of the DOSS project, with the objective of making it applicable across IoT domains and independent of any specific project tooling or platform infrastructure. It provides an overview of the current state of the relevant security testing techniques, specifies a unified methodology that defines how these techniques are combined through explicit information flows and complementary capabilities, and describes how the resulting testing evidence can be structured for integration with supply-chain artefacts such as the DSP. In doing so, the present document aims to support component manufacturers, system integrators and device operators in establishing security validation processes that are technically rigorous, lifecycle-aware and aligned with emerging regulatory and certification requirements.

1 Scope

The present document specifies a methodology for automated security testing of IoT devices. It covers third-party, open-source, and in-house software and serves as a basis for later tooling and framework implementation. The methodology integrates Static Application Security Testing (SAST), including specialized vulnerability predictions, as well as Dynamic Application Security Testing (DAST), and Interactive Application Security Testing (IAST) with a particular focus on fuzzing approaches. In addition, the present document provides a detailed description of the validation process for security patches.

2 References

2.1 Normative references

Normative references are not applicable in the present document.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act).

[i.2] DOSS Deliverable D3.1: "Unified component testing methodology", v1.0, 2024.

[i.3] DOSS Deliverable D2.2: "Integrated IoT supply chain concept and architecture", 2024.

[i.4] A. Takanen et al.: "Fuzzing for Software Security Testing and Quality Assurance", 2018.

[i.5] R. Baldoni et al.: "A survey of symbolic execution techniques", ACM Computing Surveys, 2018.

[i.6] M. Siavvas et al.: "Static-analysis-based approaches for secure software development", 2018.

[i.7] ETSI TR 104 285: "Methods for Testing & Specification (MTS); Device Security Passport".

[i.8] R. Seacord: "Secure Coding in C and C++", Addison-Wesley Professional, 2013.

[i.9] MISRA C:2025: "Guidelines for the use of the C language in critical systems", MISRA Limited, 2025.

[i.10] MISRA C++:2023: "Guidelines for the use of the C++17 language in critical systems", MISRA Limited, 2023.

[i.11] D. Davidson et al.: "Fie on firmware: Finding vulnerabilities in embedded systems using symbolic execution", Proceedings of the 22nd USENIX Conference on Security, Berkeley, 2013.

[i.12] K.-K. Ma et al.: "Directed symbolic execution", Static Analysis, pp. 95-111, 2011.

[i.13] N. Corteggiani et al.: "Inception: system-wide security testing of real-world embedded systems software", 27th USENIX Security Symposium, 2018.

[i.14] Y. Shin et al.: "An empirical model to predict security vulnerabilities using code complexity metrics", Proceedings of the Second ACM-IEEE™ international symposium on Empirical software engineering and measurement, pp. 315-317, 2008.

[i.15] I. Chowdhury et al.: "Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities", Journal of Systems Architecture, vol. 57, no. 3, pp. 294-313, 2011.

[i.16] R. Scandariato et al.: "Predicting vulnerable software components via text mining", IEEE™ Transactions on Software Engineering, vol. 40, no. 10, pp. 993-1006, 2014.

[i.17] Y. Pang et al.: "Predicting vulnerable software components through deep neural network", Proceedings of the 2017 International Conference on Deep Learning Technologies, pp. 6-10, 2017.

[i.18] I. Kalouptsoglou et al.: "Examining the capacity of text mining and software metrics in vulnerability prediction", Entropy, vol. 24, no. 5, 2022.

[i.19] J. Devlin et al.: "Bert: Pre-training of deep bidirectional transformers for language understanding", arXiv preprint arXiv:1810.04805, 2018.

[i.20] J. Zhang et al.: "An empirical study of automated vulnerability localization with large language models", arXiv preprint arXiv:2404.00287, 2024.

[i.21] I. Kalouptsoglou et al.: "Cross project vulnerability prediction based on software metrics and deep learning", International Conference on Computational Science and Its Applications. Springer, 2020.

[i.22] OWASP® Foundation: "OWASP Web Security Testing Guide (WSTG) v4.2", The Open Web Application Security Project, 2020.

[i.23] V. J. M. Manès et al.: "The Art, Science, and Engineering of Fuzzing: A Survey", IEEE™ Transactions on Software Engineering, vol. 47, no. 11, pp. 2312-2331, 2021.

[i.24] Federal Office for Information Security: "Fuzzing Primer. A Fuzz Testing Introduction with CC-specific Guidance", Version 1.6, 2020.

[i.25] M. Böhme, V.-T. Pham and A. Roychoudhury: "Coverage-based Greybox Fuzzing as a Markov Chain", in Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1032-1043, 2016.

[i.26] M. Muench, J. Stijohann, F. Kargl, A. Francillon and D. Balzarotti: "What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices", in Proc. Network and Distributed Systems Security Symposium (NDSS), 2018.

[i.27] J. Zaddach, L. Bruno, A. Francillon and D. Balzarotti: "AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares", in Proc. Network and Distributed Systems Security Symposium (NDSS), 2014.

[i.28] D. D. Chen, M. Woo, D. Brumley and M. Egele: "Towards Automated Dynamic Analysis for Linux-based Embedded Firmware", in Proc. Network and Distributed Systems Security Symposium (NDSS), 2016.

[i.29] A. Fasano et al.: "SoK: Enabling Security Analyses of Embedded Systems via Rehosting", in Proc. ACM Asia Conference on Computer and Communications Security (AsiaCCS), pp. 687-701, 2021.

[i.30] K. Serebryany, D. Bruening, A. Potapenko and D. Vyukov: "AddressSanitizer: A Fast Address Sanity Checker", in Proc. USENIX Annual Technical Conference (ATC), pp. 309-318, 2012.

[i.31] E. J. Schwartz, T. Avgerinos and D. Brumley: "All You Ever Wanted to Know about Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask)", in Proc. IEEE™ Symposium on Security and Privacy (S&P), pp. 317-331, 2010.

[i.32] N. Nethercote and J. Seward: "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation", in Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 89-100, 2007.

[i.33] M. Tappler, B. K. Aichernig and R. Bloem: "Model-Based Testing IoT Communication via Active Automata Learning", in Proc. IEEE™ International Conference on Software Testing, Verification and Validation (ICST), pp. 276-287, 2017.

[i.34] A.E. Eiben and J.E. Smith: "Introduction to Evolutionary Computing", 2nd edition, Springer, Berlin, Heidelberg, 2015.

[i.35] R. Barakat et al.: "Interactive Application Security Testing with Hybrid Fuzzing and Statistical Estimators", in: CyberSecurity in a DevOps Environment, Springer, Cham, 2024.

[i.36] H. Pearce et al.: "Examining Zero-Shot Vulnerability Repair with Large Language Models," in Proc. IEEE™ Symposium on Security and Privacy (S&P), pp. 2339-2356, 2023.

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms apply:

component: software or firmware artefact that can be tested independently

EXAMPLE: A library, service, firmware image, application module.

Component Tester (CT): system that implements the unified security testing methodology specified in the present document

Device Security Passport (DSP): structured artefact or service that contains security-relevant information about a device or component across its lifecycle

EXAMPLE: SBOM/HBOM, test results, known vulnerabilities, patch history.

security patch: changes to software or firmware intended to correct a vulnerability

System Under Test (SUT): component or set of components under security testing

3.2 Symbols

Void.

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

ACL	Access Control List
AI	Artificial Intelligence
API	Application Programming Interface
ASA	Automated Static Analysis
AST	Abstract Syntax Tree
CFG	Control Flow Graph
CI/CD	Continuous Integration/Continuous Deployment
CoAP	Constrained Application Protocol
CPU	Central Process Unit
CRA	Cyber Resilience Act
CT	Component Tester
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DSP	Device Security Passport
ELF	Executable and Linkable Format
HBOM	Hardware Bill Of Materials

HIL	Hardware-In-the-Loop
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IAST	Interactive Application Security Testing
IDE	Integrated Development Environment
IoT	Internet of Things
IR	Intermediate Representation
I/O	Input and Output
JSON	Java Script Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
QEMU	Quick Emulator
RAM	Random Access Memory
REST	Representational State Transfer
SAST	Static Application Security Testing
SBOM	Software Bill Of Materials
SDLC	Software Development Life-Cycle
SOAP	Simple Object Access Protocol
SQL	Structured Query Language
STC	Supply Trust Chain
SUT	System Under Test
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
URL	Uniform Resource Locator
VP	Vulnerability Prediction
VPM	Vulnerability Prediction Model

4 Security testing overview

4.1 General

In current industrial practice, security testing of software and firmware is often fragmented and tool-driven. Many organizations deploy a static analysis tool in their development pipeline, perform some form of penetration testing or fuzzing close to release, and run vulnerability scanners during operation. While these activities can be effective in isolation, the lack of integration between them leads to several shortcomings:

- Static analysis is frequently used as a gate in continuous integration, but its findings are not consistently followed up with targeted dynamic verification. As a result, teams may struggle with large numbers of untriaged findings and may disable classes of rules to reduce perceived noise.
- Dynamic testing and fuzzing campaigns are typically scoped to selected products or interfaces and executed late in the lifecycle, when architectural changes are difficult. Moreover, insights from dynamic testing (for example, which inputs lead to crashes, which areas are hard to test) are rarely fed back into design or secure-coding practices.
- Firmware and other embedded software often receive less attention than applications running on operating systems. Limited observability and platform-specific constraints make it harder to apply existing tools, which are typically oriented towards desktop or server environments.

Patch validation is largely ad-hoc. Once a patch is released, it is common to re-run a small number of regression tests or the original proof-of-concept exploit. Alternative paths that still trigger the same underlying flaw, or closely related vulnerabilities created by the patch, are often not explored systematically.

These limitations are particularly problematic in IoT scenarios, where the attack surface is distributed over many devices, network services and backend components, and where exploitation of a single remaining vulnerability can compromise entire fleets of products. The growing emphasis on regulatory frameworks such as the European Cyber Resilience Act further increases the need for reliable, traceable and repeatable security testing processes.

The security validation methodology proposed in the present document addresses these shortcomings by defining explicit information flows between the different security analysis and by placing the security testing within a supply-chain-aware architecture. Static results are not treated as an end in themselves but as input to dynamic testing. Dynamic testing is not purely exploratory but is guided by static analysis findings and vulnerability prediction models. Patch validation is not left to manual reasoning but is driven by structured fuzzing campaigns. All these activities are mapped so that the resulting evidence can be reused throughout the lifecycle and, for example, can be added to a DSP.

4.2 Current state of SAST

4.2.0 General

Static code analysis is a well-established technique and a central part of many secure-development programmes. Modern SAST tools support multiple programming languages, enforce secure coding standards and provide rule sets for common vulnerability classes such as buffer overflows, injection, use of insecure APIs and insufficient input validation. They are commonly integrated into Integrated Development Environments (IDEs) and CI/CD pipelines, where they automatically analyse code upon check-in or at build time.

Over the past two decades, research has significantly advanced the capabilities of static analysis. Techniques such as symbolic execution, abstract interpretation, data-flow analysis and taint analysis allow deeper reasoning about code paths and data dependencies. At the same time, AI-based approaches have emerged that use machine learning and deep learning models - particularly Transformer-based Large Language Models (LLMs) - to predict which components are likely to contain vulnerabilities and to classify vulnerabilities into Common Weakness Enumeration (CWE) categories.

Despite this progress, SAST in practice still faces several challenges:

- It tends to generate a substantial number of findings, not all of which correspond to exploitable issues. Manual triage can be time-consuming, and teams are often required to prioritize which findings to address.
- It can only reason about code that is visible to the analyser. Interactions with closed-source libraries, operating systems or hardware are often modelled imprecisely, which may lead to both false positives and false negatives.
- It is usually applied with limited context about how a component will be deployed. For instance, the same piece of code may be high-risk when exposed to the internet and low-risk when only used in an internal, access-controlled system.
- It is not automatically linked to subsequent dynamic testing or patch-validation activities. In many organizations, static and dynamic testing teams work with separate tools and reporting formats, which makes it harder to trace an issue from initial detection through to dynamic confirmation and resolution.

The methodology in the present document acknowledges these strengths and weaknesses. It uses SAST as an early and broad coverage mechanism but explicitly complements it with dynamic and interactive techniques. Furthermore, it leverages AI-based Vulnerability Prediction Models (VPMs) to focus attention on parts of the code base that are more likely to be problematic. The goal is not to replace human judgment but to provide a structured process and supporting tools that make SAST more actionable and better integrated with the remainder of the security validation process.

4.2.1 Secure coding standards

Secure coding standards are rules, guidelines, and best practices that help developers write code with fewer vulnerabilities. Standards exist for many languages (e.g. Java™, C#, Python™). For IoT and embedded systems, C/C++ standards are particularly relevant.

The Software Engineering Institute at Carnegie Mellon University publishes secure coding standards for C and C++ [i.8]. The standards define rules and recommendations. Each rule has a severity, a likelihood of leading to an exploitable vulnerability, and a remediation cost, which together determine a priority level (1 - 3). This prioritization supports risk-based remediation. Recommendations are provided for additional guidance.

MISRA C [i.9] and MISRA C++ [i.10] standards, developed for safety-critical systems, are widely used in industry. The current MISRA C [i.9] standard, MISRA C:2025, defines 223 directives and rules, while the current MISRA C++ [i.10] standard, MISRA C++:2023 (targeting C++17), defines 179 guidelines. Compliance requires adherence to all mandatory guidelines, adherence to required guidelines unless a formal deviation is documented, and consideration of advisory guidelines.

4.2.2 Code review

Code review - manual and automated - is the most common static analysis practice. Manual reviews benefit from human insight but are time-consuming and require broad expertise (architecture, implementation, security). Automated reviews use tools to detect vulnerabilities and verify compliance with coding standards. These tools take source or binary inputs and produce human-readable reports. While automation encapsulates expert knowledge, tools are limited to their built-in checks and may generate many warnings, including false positives.

In the context of automated code review, the software is transformed into an in-memory representation. For source code, this may be an Abstract Syntax Tree (AST) or a Control Flow Graph (CFG). For binaries, instructions are lifted to an Intermediate Representation (IR) to support platform independence. Subsequent analyses, including control-flow and dependence analysis, type inference and checking, and program slicing, are then run over these representations, with any potential issues being reported with messages and locations.

Because static analysis does not execute code, it lacks runtime context and cannot confirm findings on its own. Validation is therefore required. False positives may arise when tools report issues that would not occur at runtime (e.g. mitigated by secure configurations of external components). False negatives occur when tools lack rules to detect certain issues.

4.2.3 Symbolic execution

Symbolic execution is a static analysis technique originally introduced for automated test generation [i.12] and increasingly used to discover bugs and vulnerabilities across domains, including API implementations. Instead of concrete values (e.g. specific integers or strings), it uses symbolic variables while exploring execution paths. It can be applied to source code, intermediate representations, and binaries.

Symbolic variables are initially unconstrained. Constraints are added at control-flow branches, and execution forks to explore alternative paths. The accumulated constraints form the path condition. When a path terminates, an constraints solver can produce concrete inputs that satisfy the path condition; these serve as test cases that drive real execution along the same path. This enables high coverage and can expose security issues.

The advantages of this approach include the generation of test input in an automated manner, the assurance of high path coverage, and the capacity to reason about complex properties when execution states are monitored. The challenges inherent to this approach are as follows:

- Path explosion:
  - each branch can double the number of paths;
  - loops and indirect jumps exacerbate growth.
- Environment modelling:
  - engines has to model types, pointers, platform registers, memory, instruction side effects, interrupts, and hardware interactions [i.13].

Two common mitigations to path explosion are:

- Dynamic symbolic execution (concolic testing):
  - analyse "interesting" instructions symbolically and others concretely;
  - optionally restrict scope via program slicing.
- Path selection:
  - prioritize or abandon paths using heuristics tailored to the domain [i.11], [i.12].

4.2.4 Taint analysis

Taint analysis detects vulnerabilities arising from improper data sanitization - when data derived from untrusted input is used in security-sensitive operations. Sources are program points at which untrusted data can be entered (e.g. environment variables, stdin). Sinks are security-sensitive operations that attackers may exploit (e.g. indirect jumps). Analysis propagates a taint flag from sources according to a propagation policy. A vulnerability is reported when tainted data reaches a sink. Note that program integrity may be compromised before a sink is executed (e.g. a tainted return address due to a buffer overflow).

Static taint analysis considers all possible paths from sources to sinks and requires control-flow and dependence information. It struggles with indirect memory accesses, indirect calls, and pointer aliasing (multiple names referencing the same memory chunk).

Key challenges include:

- Under-tainting: missing certain information flows (especially those mediated by control dependencies), which are hard to capture purely dynamically.
- Over-tainting (taint spread): tainting values that are not derived from untrusted input.

4.2.5 AI-driven static analysis

Automated Static Analysis (ASA) automates code inspection for security weaknesses by analysing structure, syntax, and logic without execution. ASA tools report potential issues (e.g. buffer overflows, exception handling errors, null dereferences, memory corruption) based on predefined rules. ASA scales to large codebases and provides consistent, thorough coverage with minimal manual effort.

However, ASA can generate many false positives. A Vulnerability Prediction (VP) mechanism can prioritize alerts by estimating the likelihood that a component is vulnerable, thereby focusing attention on higher-risk findings. Two ML-based VP approaches are common:

i) software-metrics-based (e.g. complexity, lines of code) [i.14], [i.15]; and

ii) text-mining-based, which tokenizes source code and learns textual patterns [i.16], [i.17].

The latter often performs better [i.18].

Recent NLP advances, especially Transformer-based Large Language Models (LLMs), enable transfer learning for VP [i.19].

Open challenges include choosing the optimal Transformer variant (encoder-only, decoder-only, encoder-decoder) and training configurations for VP, as studies often report final models without ablations. Another challenge is predicting not only whether a component is vulnerable but also the type of vulnerability. The CWE system categorizes software weaknesses, but expert-driven assignment can be time-consuming and lag behind disclosures [i.20].

4.3 Current state of DAST

DAST comprises techniques that exercise a running System Under Test (SUT) through its exposed interfaces and monitor its behaviour for signs of security vulnerabilities or weaknesses. In contrast to SAST, which reasons about code without execution, DAST observes concrete executions under specific inputs and environments. It is therefore well suited to detecting vulnerabilities that depend on runtime state (e.g. memory layout, timing, configuration) or on the interplay between multiple components.

In industrial practice, DAST has evolved from manual penetration testing and simple web-application scanners towards a diverse landscape that includes web and API scanning, protocol-level testing, coverage-guided robustness testing of binaries and services, firmware and Hardware-In-the-Loop (HIL) testing, as well as dynamic taint analysis and various forms of runtime instrumentation [i.23]. For IoT ecosystems, all of these aspects are relevant, because security depends not only on internet-facing web services but also on device-local interfaces, embedded protocol stacks and cloud backends that form part of the overall system.

The present clause provides an overview of the main dynamic testing approaches that are currently used or emerging for IoT-relevant software and firmware. Later clauses in the present document build on these techniques and embed them into a unified security validation methodology.

Web and API scanning

The most established form of DAST is web and API scanning [i.22]. Commercial and open-source scanners interact with running web applications, device-embedded web interfaces and HTTP-based APIs. They typically discover pages and endpoints, identify input vectors such as form fields, headers and query parameters, and then submit crafted requests that embed attack payloads. The goal is to trigger observable misbehaviour, for example database manipulation (e.g. SQL injection), reflected payloads in HTML responses (cross-site scripting) or anomalous HTTP status codes, which indicate classes of vulnerabilities such as injection, cross-site scripting, broken access control or insecure session management [i.22].

Over the last decade, these tools have evolved from simple crawlers to platforms that understand REST, SOAP and, increasingly, GraphQL and JSON-based services. They can often import interface descriptions such as OpenAPI specifications to drive systematic testing of API operations [i.22]. In the IoT domain, web and API scanners are used not only against public cloud services but also against local device management interfaces, configuration portals exposed over HTTP, and vendor portals used for provisioning and update orchestration.

The strength of web and API scanning lies in its relative ease of deployment. A scanner can be pointed at a URL or host address and can begin testing without requiring access to source code or build artefacts. At the same time, current practice shows clear limitations. Scanners typically remain black-box tools with little or no insight into internal code coverage. They tend to focus on well-known vulnerability patterns in HTTP-centric scenarios and can struggle with complex, stateful workflows such as multi-step onboarding procedures, certificate-based authentication or device pairing protocols. For IoT systems that combine web interfaces with proprietary transport protocols and constrained devices, web and API scanning therefore forms an important but incomplete part of the DAST picture.

Network- and protocol-level testing

Beyond HTTP, many IoT components communicate using specialized application protocols such as MQTT and CoAP or proprietary protocols running over TCP, UDP or serial links. Dynamic testing at this level aims to verify that protocol implementations behave robustly under malformed or unexpected traffic, and that their state machines enforce the intended security properties [i.33].

In current practice, testers frequently adopt a mutation-based strategy: they capture representative protocol traffic, for example during normal device-to-cloud communication, and then generate variants by perturbing message fields, lengths, sequence numbers or timing. By observing how the SUT reacts to these perturbed messages, issues such as unhandled exceptions, protocol desynchronization, missing bounds checks or failure to enforce authentication steps can be detected. For some widely used protocols, model-based or grammar-based tools are available. These tools rely on a formal or semi-formal description of the protocol syntax and state machine and generate sequences of valid but semantically unusual messages that drive the implementation into corner cases [i.33].

In IoT deployments, protocol-level testing is applied both at the network edge and in backend infrastructure. Device firmware and gateways are tested for resilience against malformed sensor data and control messages, while cloud brokers and message-processing services are tested for their handling of arbitrary payloads, topic hierarchies and quality-of-service settings. A particular challenge is the setup of realistic end-to-end testbeds in which devices, gateways, brokers and security services such as public key infrastructures operate in realistic configurations. Without such testbeds, certain state-dependent and timing-dependent vulnerabilities remain hard to expose.

Coverage-guided robustness testing and fuzzing

Coverage-guided fuzzing has emerged as a powerful technique for discovering implementation-level vulnerabilities [i.23], [i.25]. Mutation-based fuzzers instrument the SUT at compile-time or binary-rewriting stage to collect lightweight execution feedback, such as basic-block or edge coverage. During testing, they maintain a corpus of inputs and iteratively mutate those inputs that cause new coverage [i.26]. Over time, this evolutionary process drives the SUT through an increasing number of execution paths, including paths that developers may not have anticipated. Crashes, assertion failures and violations detected by runtime sanitizers are recorded as findings.

In today's software industry, coverage-guided fuzzers are used extensively for parsers (for example for image, media and configuration formats), protocol stacks, compression and cryptographic libraries and background services [i.23].

Generation-based fuzzers complement this by using explicit grammars or models to construct syntactically correct test cases that exercise semantic properties. Hybrid approaches combine both strategies [i.23].

The practical relevance of coverage-guided fuzzing is further reflected in certification and evaluation guidance. In Germany, the Federal Office for Information Security (BSI) provides the AIS 50 Fuzzing Primer [i.24], which establishes a structured methodology for the planning, execution and assessment of fuzzing activities in security evaluations.

For IoT components, coverage-guided robustness testing is applicable wherever an entry point can be defined that consumes externally controllable input. This may be a function that parses a configuration file, a routine that decodes a network frame or a library that processes commands received from a gateway. In many cases, such testing is performed off-device, for example by compiling the relevant component for a host platform with instrumentation enabled [i.4]. The current state of practice shows that when suitable harnesses are available, coverage-guided fuzzing can reveal subtle memory-safety issues and logic flaws that would be very difficult to identify through manual inspection or traditional black-box testing [i.25]. However, constructing and maintaining these harnesses requires non-trivial engineering effort, especially when dealing with complex initialization sequences or dependencies on external components.

Firmware and hardware-in-the-loop dynamic testing

Testing embedded firmware and low-level device software dynamically is more challenging than testing desktop applications [i.27]. Firmware typically runs without a full operating system, interacts with hardware through memory-mapped I/O and interrupt handlers, and may rely on precise timing. Two main approaches are visible in current practice.

The first approach is Hardware-In-the-Loop (HIL) testing. Here, actual devices or prototype boards are exercised through their physical interfaces, for example Ethernet or Wi-Fi®. Automated test systems send crafted inputs, simulate environmental conditions and observe outputs such as log messages, serial debug output or sensor values. HIL testing comes close to real deployment conditions and is therefore valuable for capturing timing- and power-related effects. Its limitations include limited observability of internal program state, slower test execution speeds and difficulties in managing and resetting large device fleets during extensive campaigns.

The second approach is rehosting, where firmware images are executed inside an emulator that models the target CPU architecture and sufficient parts of the hardware environment [i.28], [i.29]. Research prototypes and emerging industrial tools use frameworks based on QEMU or custom simulators to load firmware, emulate RAM and peripherals, and run the code under test [i.28]. Within such rehosting environments, it becomes possible to combine dynamic testing with instrumentation similar to that used in coverage-guided fuzzing and memory-sanitizer frameworks [i.27], [i.29]. Memory accesses to static, stack and heap regions can be monitored and checked against redzones and access-control lists to detect out-of-bounds writes and use-after-free errors, even when they do not immediately lead to device crashes [i.27].

While rehosting is conceptually powerful, its practical use in industry is still relatively limited [i.29]. Creating accurate peripheral models for diverse microcontrollers and system-on-chip platforms is labour-intensive, and discrepancies between the emulated and real hardware require careful management [i.27], [i.29]. As a result, many organizations still rely on a combination of manual HIL testing and selective rehosting for particularly critical components.

Dynamic taint analysis and runtime instrumentation

Dynamic taint analysis aims to follow the flow of data originating from untrusted sources, such as network inputs, environment variables or user-supplied parameters, as the program executes [i.31]. Values derived from these sources are marked as tainted, and the analysis reports a potential vulnerability when tainted data influences security-critical operations such as control-flow decisions, pointer values or system calls [i.31]. In theory, this provides a powerful means of detecting inadequate input validation and sanitization.

In practice, dynamic taint analysis incurs significant runtime overhead and requires sophisticated tooling [i.31], [i.32]. For this reason, it is most frequently used in academic research, specialized security assessments or focused campaigns where particular data flows need to be understood in depth. When combined with fuzzing, taint analysis can help identify which parts of the input most strongly affect program behaviour, thereby guiding more effective mutations [i.23].

More widely adopted in industry are lighter-weight forms of runtime instrumentation [i.30]. Compilers and binary-instrumentation frameworks offer sanitizer modes that inject checks for memory-safety violations, undefined behaviour and other classes of errors [i.32]. Address-sanitizer-style tools track the use of heap and stack buffers and report overflows, use-after-free, double frees and related issues [i.30]. Undefined-behaviour sanitizers detect constructs such as integer overflows, invalid pointer arithmetic or violations of language rules that may have security consequences.

For IoT components, it is common practice to build host-side or emulated versions of firmware and libraries with such instrumentation enabled and to run them under test workloads, including fuzzing and protocol-level tests [i.27], [i.30]. This approach significantly improves defect observability and reduces the risk that subtle corruptions go unnoticed. On resource-constrained production devices, such instrumentation is usually disabled due to performance and memory constraints, but the defects it exposes during testing can be fixed before deployment.

Hybrid, model-based and AI-assisted dynamic testing

Hybrid testing approaches combine two or more dynamic or static-dynamic techniques to overcome the individual limitations of each. The most prominent example is the combination of coverage-guided fuzzing with concolic or symbolic execution. Pure fuzzing can struggle to pass complex input conditions that guard deep code paths (e.g. multi-byte magic values, checksums, or cryptographic tokens), while symbolic execution scales poorly to large programs. Hybrid engines address this by applying symbolic execution selectively when fuzzing stalls, generating new seeds that satisfy difficult path conditions and feeding them back into the fuzzer corpus. This combination achieves substantially higher path coverage than either technique alone.

Model-based and grammar-based dynamic testing takes a complementary direction. Instead of relying solely on feedback from the SUT, the test generator uses an explicit model of the protocol or input format to produce syntactically and semantically meaningful test cases [i.33]. For IoT communication protocols, active automata-learning techniques can infer a state-machine model of a running implementation and then generate sequences of messages that drive it through all discovered states, including those that are rarely exercised in normal operation [i.33]. This approach is particularly valuable when testing for logical vulnerabilities such as authentication-bypass or incorrect state-transition handling that are difficult to reach through purely random mutation.

AI-assisted dynamic testing is an emerging area in which machine-learning models are used to guide test generation or to prioritize which inputs are most likely to expose vulnerabilities [i.23]. Approaches include neural networks trained to predict code-coverage feedback from input features, reinforcement-learning agents that learn mutation strategies over time, and large language models used to generate plausible protocol messages or configuration inputs. While the field is still maturing and many proposed techniques have been evaluated mainly on benchmark suites, early results suggest that AI assistance can improve fuzzer efficiency, especially in domains with structured inputs such as scripting engines, configuration parsers and network protocol implementations [i.23].

For IoT security validation, the most immediately applicable hybrid techniques are those that combine coverage-guided fuzzing with selective concolic execution for hardened parsing routines, and model-based protocol testing for validating state-machine correctness of communication stacks. AI-assisted approaches are expected to become more relevant as the tooling matures and as training datasets specific to IoT firmware and protocols become available.

5 Specification of a security validation methodology

5.1 General

The security validation methodology described in the present document is based on the idea that no single testing technique is enough on its own to provide adequate assurance. Modern IoT products are built from a large number of heterogeneous software and hardware artefacts, including self-developed code, third-party libraries and open-source components. These artefacts are updated independently, reused in different products and often originate from suppliers with varying levels of maturity. At the same time, IoT devices frequently operate in constrained environments and remain deployed for long periods, which makes it difficult to correct design-time security issues once devices are in the field.

By integrating complementary testing methods, a security validation methodology addresses vulnerabilities from multiple perspectives and supports both self-developed and third-party components. SAST approaches identify potential weaknesses in source code without executing it and is therefore well suited for early detection of weaknesses during the design and implementation phases. This enables early detection within the Software Development Life Cycle (SDLC) and supports secure-by-design practices. DAST evaluates system behaviour during runtime without requiring access to source code. It is particularly valuable when only binaries or firmware are available. IAST combines static and dynamic techniques by using static findings to drive dynamic tests and using dynamic results to confirm or refute static analysis findings. Finally, dedicated patch-validation activities focus on assessing whether security patches remove vulnerabilities and do not introduce residual or new weaknesses.

This clause specifies the recommended set of security testing techniques and how they are combined into a coherent methodology. The methodology defines how complementary methods are orchestrated so that each compensates for the blind spots of the others and the overall effectiveness of security validation is increased [i.2].

The following subclauses describe, in a process-oriented manner, how:

- SAST is used as an entry point to generate structured security hypotheses.
- DAST exercises running components with a focus on robustness and exploitability.
- IAST links static and dynamic evidence through targeted verification.
- Firmware-oriented fuzzing extends dynamic testing to embedded and bare-metal contexts.

The intent is that implementations select and tailor concrete tools according to their environment, while adhering to the information flows and capabilities defined in this clause to achieve efficient and comprehensive security testing [i.2] and [i.3].

5.2 Static Application Security Testing (SAST)

SAST is the first stage of the methodology to detect potential vulnerabilities as early as possible, ideally during development before a component is deployed or integrated into a larger system [i.6]. For IoT components, this early detection is particularly important because devices may be difficult or expensive to update once deployed and because vulnerabilities in widely reused components can affect many products simultaneously.

In the methodology, SAST is understood as a combination of two complementary activities. The first is Automated Static Analysis (ASA), which uses rule-based static analysis tools and language-specific scanners to detect coding errors and violations of secure coding standards [i.6], [i.8], [i.9]. The second is AI-driven Vulnerability Prediction (VP), which uses machine learning and deep learning models [i.19] to estimate which components are more likely to contain vulnerabilities and to categorize those vulnerabilities where possible [i.20]. Together, these activities produce a prioritized list of candidate issues that can then be further examined through IAST and DAST.

The SAST workflow is structured as a sequence of well-defined steps, from acquisition of input artefacts to export of results. Each step has a clearly identified role and interacts with the others through explicit information flows.

The starting point is the set of artefacts available for a given component. Ideally, this includes the full source code of the component, build scripts and configuration files. From a DSP or other STC elements [i.7], [i.3], metadata such as component type, programming languages, known third-party libraries, security requirements and exposure scenarios can be retrieved. This contextual information can influence the choice of rule sets and the severity scaling of findings.

Once the artefacts and context have been collected, Automated Static Analysis will be performed [i.6]. In a typical implementation, a central platform orchestrates the analysis. In this instance, it is recommended that the configuration be set to apply relevant rule profiles or secure-coding rule sets based on the retrieved information, such as the programming language [i.8], [i.9]. The platform then invokes language-specific analysers, parses the code into intermediate representations (such as abstract syntax trees and control-flow graphs) and applies static checks to identify potential defects and rule violations.

However, rule-based static analysis alone is not sufficient. To address this, the methodology incorporates AI-based Vulnerability Prediction [i.17], [i.18], [i.21]. At this stage, code fragments - typically entire files or functions - are converted into a format suitable for consumption by a Vulnerability Prediction Model. Depending on the model, this may involve tokenizing the source code, normalizing identifiers and constants, and mapping tokens to embeddings. Here, one or more Transformer-based models that have been pre-trained on large corpora of programming languages and then fine-tuned on labelled datasets of vulnerable and non-vulnerable code can be used. For each analysed code fragment, the model should provide a score representing the likelihood that the fragment contains a vulnerability.

In addition to binary predictions of vulnerability likelihood, some models also provide a predicted vulnerability class, for example a CWE identifier [i.20]. These predictions are not considered definitive but help orient subsequent analysis and reporting. For example, if a code region is predicted to be vulnerable and associated with CWE-119 (buffer overflow) [i.8], it may be a candidate for particular fuzzing strategies that focus on input length and boundary conditions [i.4].

For each static finding (results of ASA and VP), it can be determined whether the surrounding code is in a region with high, medium or low vulnerability likelihood, and it may attach a predicted vulnerability type. Findings in high-likelihood regions, particularly those with severe rule violations, are given higher priority for dynamic verification and remediation [i.2], [i.6]. Conversely, findings in low-likelihood regions may be marked as lower priority, while still being recorded. This correlation step transforms a flat list of static warnings into a structured set of security hypotheses that can be systematically followed up by other parts of the methodology.

Finally, the SAST results will be reported. The reported information typically includes a component identifier, the list of static issues with their locations and severities, the vulnerability prediction scores and classes [i.18], [i.20], and any additional context such as critical lines and path summaries.

5.3 Dynamic Application Security Testing (DAST)

Dynamic tests complements SAST by examining the behaviour of components during execution [i.23]. While SAST is effective at detecting certain patterns and design-time issues, only dynamic testing can reveal vulnerabilities that depend on runtime state, specific sequences of inputs or interactions with external systems [i.4], [i.23]. In the present methodology, DAST is centred around fuzzing (or fuzz testing), both for general applications and for embedded firmware.

The central idea of fuzzing is to run the SUT under a large number of automatically generated input scenarios and observe whether any of these provoke incorrect behaviour such as crashes, hangs, assertion failures or violation of security invariants [i.4]. Fuzzers differ in how they generate inputs and in how much information they use about the internal state of the SUT. Black-box fuzzers treat the SUT as opaque and generate inputs based solely on the interface specification [i.23]. White-box fuzzers use symbolic execution or constraint solving to derive inputs that satisfy complex path conditions [i.5], [i.12]. Grey-box fuzzers take an intermediate approach, using lightweight instrumentation to obtain coverage feedback and drive a genetic search algorithm towards unexplored code [i.25].

The objective of DAST is to uncover vulnerabilities inherent in the SUT. A primary technique is fuzz testing, a well-established method that stimulates the SUT with malformed or unexpected inputs to reveal weaknesses [i.4]. Fuzzing aims to expose flaws that attackers could exploit and to surface stability issues [i.23]. By systematically generating inputs that deviate from normal operation, fuzzers can trigger unexpected behaviours, crashes, or security breaches, enabling developers to identify weak points in the code.

Fuzzing is commonly categorized as black-box, white-box, or grey-box [i.23]. Black-box fuzzing operates without internal knowledge of the target, making it suitable for third-party components [i.4]. White-box fuzzing leverages source-level insight to craft targeted tests that can expose deeper issues [i.5]. Grey-box fuzzing balances both, using limited internal signals (e.g. coverage) to guide the search [i.25]. Such guidance is often implemented with genetic algorithms [i.34].

Genetic algorithms are optimization techniques inspired by natural evolution [i.34]. They iteratively improve a population of candidate solutions ("individuals"). Each generation is evaluated with a problem-specific fitness function; fitter individuals are selected to produce the next generation through crossover (recombination) and mutation (random variation) [i.34]. The process repeats until a stopping criterion is met (e.g. generation limit or fitness threshold).

One of fuzzing's key strengths is automation, which enables extensive testing far faster than manual efforts [i.4], [i.23]. Fuzzing can be integrated into the software development lifecycle to detect issues early and improve overall quality [i.2]. As systems grow more complex and interconnected, fuzzing remains essential for enhancing security and robustness [i.23].

Fuzzing goals vary: maximizing code coverage, targeting specific code regions, or finding particular vulnerability classes [i.23]. Many modern approaches emphasize coverage growth and use diverse techniques to reach hard-to-exercise paths guarded by complex conditions or deep nesting [i.25].

Multiple fuzzing techniques can be employed to comprehensively test components [i.23]. The aim is thorough evaluation for both general robustness and specific vulnerability classes. Coverage-driven strategies are combined with vulnerability-targeted techniques to improve efficiency [i.25]. Custom fuzzing goals can target weaknesses that standard fuzzers may miss [i.4]. Fuzzing is applied broadly to the SUT and also used for specific tasks such as validating static-analysis findings (see clause 5.3) and verifying security patches (see clause 6).

This multifaceted strategy - combining different fuzzing modes, enhancing existing tools, and developing tailored solutions - supports rigorous testing [i.2]. The result is improved reliability and security, with higher confidence that potential vulnerabilities are identified and addressed.

5.4 Interactive Application Security Testing (IAST)

5.4.1 General

IAST in the context of this methodology refers to the coordinated use of static and dynamic techniques [i.35]. Rather than running SAST and DAST in isolation, IAST establishes a feedback loop in which static findings and vulnerability predictions guide dynamic testing, and dynamic observations refine the understanding of static findings [i.35]. The objective is to reduce false positives, concentrate effort on the most relevant issues and generate stronger evidence for confirmed vulnerabilities [i.2], [i.35].

The starting point for IAST is the set of findings and predictions produced by the SAST stage [i.6], [i.35]. For each component, a list of static issues, each annotated with severity, location, rule identifier and possibly a predicted vulnerability type [i.20], as well as vulnerability-likelihood scores at file or function level [i.18] is provided. This information can be analysed to select a subset of targets for IAST. A target may be a specific static finding, a particular function or a region of code that is considered security-critical.

Selection criteria can include the severity of the static finding, its location (for example, whether it is in a parser or an authentication routine), the vulnerability-likelihood score from the VP model, [i.18] and contextual information from the DSP [i.7] (such as whether the component processes unauthenticated external inputs). The aim is to assemble a manageable set of targets that, if explored dynamically, will yield a substantial improvement in confidence [i.35].

5.4.2 Verification of SAST findings

Verification of SAST findings proceeds in two main ways, depending on the nature of the static information available [i.35].

In cases where the static analysis has produced explicit path constraints leading to a suspected vulnerability - for instance - it could be useful to use constraint solving to generate dedicated test cases [i.5], [i.12], [i.12]. Therefore, the relevant conditions and operations on the input are translated into a form acceptable to a constraint solver [i.5]. The constraint solver is then used to ascertain the existence of inputs that satisfy the collected constraints system. In addition to the path constraints, it is important to add the specific constraints that the variables need to fulfil to trigger the suspected vulnerability. For instance, the given length has to be larger than the accurate buffer size for a buffer overflow vulnerability [i.8] to be triggered. If the solver confirms that the constraints are satisfiable, it produces concrete input values [i.5], [i.12]. Then test cases can be constructed and executed against the SUT. If the execution reaches the code location indicated by SAST and exhibits the vulnerability (such as memory violation, assertion failure, or security policy breach), the finding is treated as confirmed. If the solver indicates that the constraints are unsatisfiable for all possible paths, this suggests that the suspected vulnerability reported by the static analysis is more likely a false positive [i.6]. However, because constraint solving relies on models that may not fully capture the behaviour of external components, directed fuzzing can be used for further exploration [i.23], [i.25], [i.35].

In practice, however, there are also cases where the constraint solver does not return a useful answer within acceptable resource limits (e.g. due to timeouts, excessively large or highly non-linear constraint systems, or imprecise models of external components) [i.5], or where the constraints are considered too complex to be handled reliably. In all of these situations - unsatisfiable, unknown, timeout, or otherwise not tractable - the methodology employs directed fuzzing as a fallback mechanism [i.25], [i.35]. Directed fuzzing drives exploration towards the target region to explore alternative execution paths that may bypass the limitations of the constraint model [i.23], [i.35]. As test cases are generated and executed, it will be monitored whether the target region is reached and whether the suspected vulnerability occurs. This ensures that suspected vulnerabilities are still investigated dynamically, even when constraint solving alone cannot provide definitive evidence [i.35].

Figure 1 depicts the IAST process workflow for the verification of SAST findings. Starting from a static finding, the diagram shows the two main branches using constraint solving and directed fuzzing. It illustrates how path constraints are turned into targeted test cases, how fuzzing is used as a fallback or complement, and how the dynamic results are classified into confirmed vulnerability, likely false positive, and inconclusive.

The outcome of verification is recorded in three possible states: confirmed vulnerability, likely false positive and inconclusive [i.35]. A finding is considered confirmed if dynamic testing reaches the indicated code region and triggers a failure consistent with the static finding. It is considered likely false positive if extensive directed fuzzing achieves high coverage of relevant code but fails to trigger the suspected behaviour, especially when constraint solving has shown relevant paths to be unsatisfiable [i.5]. It remains inconclusive when neither condition is met, for example because the code is difficult to reach or because available time was insufficient [i.35]. For inconclusive findings, the extent of dynamic exploration (number of test cases, coverage of relevant functions) is reported so that human analysts can assess residual risk.

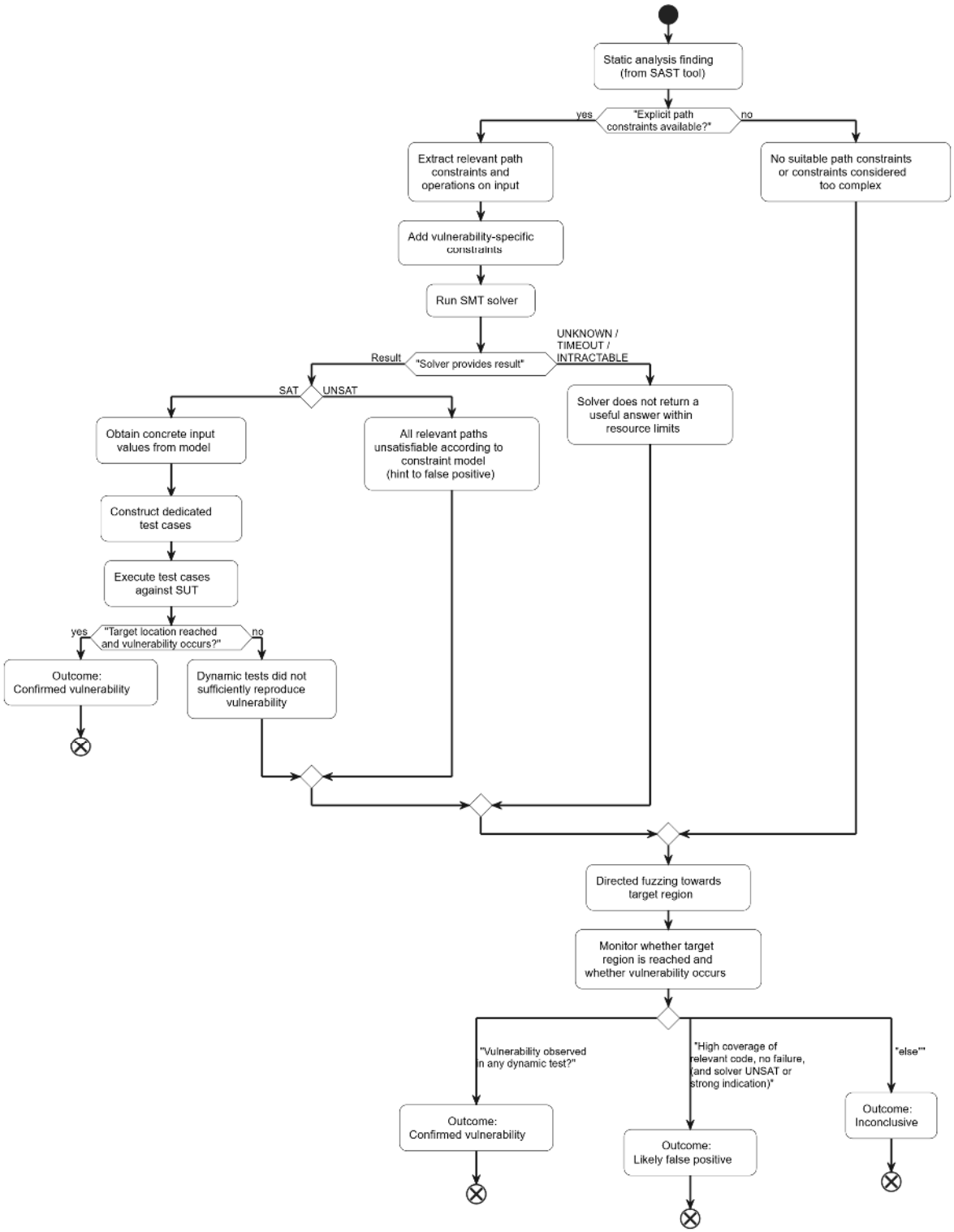


Figure 1: IAST process flow diagram for the verification of SAST findings

5.5 Firmware Fuzzing

5.5.0 Firmware Fuzzing Introduction

Firmware fuzzing targets embedded software that runs directly on hardware or on a minimal runtime environment [i.26], [i.27]. Such software often interacts with peripherals through memory-mapped I/O, handles interrupts and may not rely on an operating system [i.26]. When planning dynamic testing in this context, it needs to be considered that executing the firmware directly on physical devices may limit observability and control [i.26], [i.29], and that standard fuzzing tools typically assume a process model that is not available on bare-metal platforms [i.27].

These challenges can be overcome by rehosting [i.28], [i.29]. The firmware is executed in an emulator such as QEMU that models the target CPU architecture and implements enough of the hardware environment to allow meaningful execution [i.28]. QEMU (or a similar emulator) should be configured to load the firmware image, emulate memory regions, handle interrupts and simulate peripheral behaviour [i.28], [i.29]. Similar to that used in general fuzzing [i.23], [i.25], fuzzing instrumentation can be embedded on top of this execution environment, but adapted to the specifics of the firmware [i.27]. When doing so, it is important to ensure that the instrumentation does not interfere with timing-sensitive behaviour more than necessary [i.26].

A key objective in firmware fuzzing is to detect memory-safety violations not only when they lead to fatal hardware exceptions (such as hard faults) but also when they corrupt internal state in more subtle ways [i.26], [i.30]. To achieve this, it is useful to treat three memory domains separately:

- static memory holding global variables;
- stack memory for local variables and call frames; and
- heap memory used for dynamic allocation [i.30].

5.5.1 Static Memory containing global variables

For global variables stored in static memory, a combination of static analysis and runtime instrumentation can be used [i.27], [i.30]. Provided that symbol information is available, as is typically the case for ELF files [i.27], a static analysis tool can be used to identify global variables and locate all instructions that access them. For each variable, a list of code locations (usually program counters) is created that legitimately read or write it. This list serves as an Access Control List (ACL) for that variable [i.27]. This static analysis can be reduced in precision by stripped binaries or aggressive compiler optimizations [i.29].

During firmware execution under the emulator, every access to the static memory region can be intercepted [i.28]. The instrumentation checks whether the access is performed by an instruction on the ACL for the corresponding variable [i.27]. If an access originates from a location that is not authorized for that variable, it is flagged as a potential memory-safety violation [i.30]. Such violations may indicate buffer overflows in which an array is indexed beyond its bounds and neighbouring global variables are overwritten [i.8], [i.30]. In practice, the balance has to be struck between the granularity of this checking and the runtime overhead it introduces [i.29].

In many firmware designs, global variables are not isolated but appear as fields of large structures, which are then passed to functions by reference. For such cases, static ACLs become less effective, because many accesses go through pointers whose aliasing cannot be fully resolved statically [i.29]. To increase the likelihood of detecting out-of-bounds accesses in these situations, binary instrumentation can be performed to insert 'red zones' between symbols [i.30]. One approach is to relocate the entire RAM region to a non-conflicting part of the address space, insert padding between global variables and generate a mapping from original addresses to relocated addresses. Code references to global variables are adjusted to point into the relocated region, and the emulator is extended so that any access to the original, now unmapped, region is treated as an error [i.28]. When implementing this scheme, care needs to be taken not to collide with memory-mapped I/O regions and to correctly update all relevant pointers [i.27], [i.29]. This combination of ACL enforcement and redzone instrumentation can significantly improve visibility into static memory violations [i.30].

5.5.2 Stack and Heap sanitization

The aim of stack sanitization is to detect stack-based buffer overflows and frame corruptions [i.30]. Therefore, the emulator can be extended to track the current call stack by observing changes to the program counter, the link register, and the stack pointer, as well as using symbol information where available [i.27]. When a function is called, a new stack frame is created in concept; when it returns, the frame is destroyed. Red zones are placed between stack frames by adjusting the stack pointer and reserving padding [i.30]. During execution, any memory access that crosses into a red zone is treated as an indication that the code has written beyond the bounds of a local variable or otherwise corrupted the stack [i.30]. When designing this mechanism, account should be taken of the calling conventions and optimization strategies of the target architecture, since these influence how reliably frames can be identified [i.29].

Heap sanitization follows a similar principle but focuses on dynamically allocated memory [i.30], [i.32]. Calls can be intercepted to allocation and deallocation functions such as malloc and free, either through direct hooking of library functions or through instrumentation [i.32]. When memory is allocated, it is recommended to surround the allocated block with redzones and to record its bounds in a heap metadata structure [i.30]. When memory is freed, mark the block as invalid [i.30]. Every subsequent access to heap memory is then checked against this metadata: accesses that touch redzones or freed blocks are flagged as out-of-bounds writes or use-after-free errors [i.30], [i.32]. To avoid blind spots in this checking, it is important to identify all allocator entry points used by the firmware, including custom allocators [i.29].

Such a firmware fuzzing setup can reveal a broad range of memory-safety vulnerabilities by combining static memory, stack and heap sanitization [i.30] with coverage-guided fuzzing [i.23], [i.25] in the emulated environment [i.27], [i.28]. These include issues that may not immediately result in hard faults, but which could still be exploited by attackers to corrupt program state or hijack control flow [i.26], [i.30].

6 Validation of security patches

6.1 Introduction

Validating security patches is a different task to the initial validation of a system. Security patch validation requires a vulnerability to have been detected and patched beforehand. Therefore, the main goal here is not to find new vulnerabilities, but to validate that the applied patch closes the vulnerability.

Patch validation is a process which aims to verify the completeness and effectiveness of a security patch [i.2], [i.25]. In practice, patches may only partially remediate a vulnerability, for example by addressing a single crashing path while leaving alternative execution paths exploitable, or by fixing one data condition but not others [i.25]. Attackers can study published advisories, code changes or binary diffs and then craft new inputs or interaction sequences that still exercise the original flaw [i.36]. Patch validation therefore aims to search systematically for such residual exploit paths before the patched software is released [i.2].

A typical example is a double-free vulnerability. Consider a function that allocates memory, calls a processing routine and then calls an error-handling routine, before finally freeing the memory. If the processing routine also frees the same allocation, a double free arises [i.30]. A patch that simply removes the free in the processing routine may eliminate the originally reported crash, but if the error-handling routine still frees the allocation on certain paths, the same underlying defect can be triggered via a different control-flow path [i.25]. Without explicit validation, such residual paths can remain unnoticed and provide attackers with a straightforward way to revive the issue [i.36].

6.2 Genetic grey-box fuzzing for security patch validation

The patch-validation methodology is based on genetic grey-box fuzzing [i.25]. The central idea is to deliberately generate many different inputs that trigger the original bug on the unpatched version, but along as many distinct execution paths as possible [i.25], [i.34]. This diverse suite of crashing inputs is then replayed on the patched version in order to assess whether the patch closes all of these paths [i.2].

The process assumes that the following information is available:

- an unpatched version in which the vulnerability can be reproduced;
- a patched version that claims to fix the vulnerability;

- at least one input (for example from an earlier fuzzing campaign or a reported exploit) that triggers the vulnerability on the unpatched version;
- a set of critical lines in the source code that are essential for triggering the vulnerability (for example the allocation and deallocation sites in a double-free, or the read and write operations around a buffer in an overflow).

Starting from the known crashing input on the unpatched version, a genetic grey-box fuzzer evolves new inputs. Each candidate input is executed, and runtime instrumentation records which source lines are covered, whether the critical lines are hit, and whether a crash occurs. A dedicated fitness function assigns a numerical score to each [i.25], [i.34]. This fitness function combines three aspects:

- First, the number of critical lines executed. Inputs that cover more of the lines known to be essential for triggering the bug are scored higher. This encourages executions that drive the program through all key operations associated with the vulnerability.
- Second, the amount of new coverage beyond the original crash path. When the original crashing input is executed, the full set of covered lines is recorded as the baseline crash coverage. New inputs receive additional fitness when they exercise lines outside this baseline set. This drives the search towards alternative paths that still pass through the critical region, rather than simply rediscovering the original route.
- Third, the occurrence and localization of crashes. Certain vulnerabilities depend not only on which lines are executed but also on input-specific data conditions, such as buffer lengths or index values. The fitness is therefore increased when an input triggers a crash of the relevant type (for example a double free or buffer overflow). If the crash occurs at the same source line as the original vulnerability, the score is increased further, because this indicates that the same underlying defect has been exercised, albeit potentially via a different path.

These three components are combined into a single scalar fitness value (for example by summation, possibly with normalization or weighting) [i.34]. A genetic algorithm [i.34] then selects, mutates and recombines inputs according to their fitness. Over successive generations, this process tends to produce a pool of inputs that all trigger the target vulnerability, but along varied execution paths and with varied data patterns [i.25]. This pool forms a stress-test suite specifically tailored to the vulnerability under consideration.

During fuzzing of the unpatched version, each execution produces a trace of covered lines and an indication of whether a crash has occurred. The trace is analysed to compute the fitness of the input based on critical-line coverage, additional coverage beyond the baseline crash path and crash characteristics [i.25]. Inputs with high fitness are retained and used as parents for the next generation; they are mutated to explore neighbouring regions of the input space [i.34]. Inputs with low fitness are discarded. This loop continues until a stopping criterion is reached, for example a maximum number of generations, a plateau in fitness improvements or a predefined time budget [i.23], [i.34].

Once a sufficiently diverse suite of crashing inputs has been collected on the unpatched version, these inputs are systematically replayed on the patched version under the same instrumentation. For each input, the validation observes whether the critical lines are still executed and whether a crash of the target type occurs. Several outcomes are possible:

- If none of the previously crashing inputs causes the patched version to crash, and the traces show that the critical lines are still reached, this provides strong evidence that the patch successfully removes the vulnerability along all explored paths. Residual risk cannot be entirely excluded, but the likelihood of exploitation is significantly reduced.
- If some inputs still trigger the same type of crash, especially at or near the original vulnerable line, this indicates that the patch is incomplete. It has not removed all execution paths or data conditions leading to the vulnerability. These residual inputs and their traces serve as concrete evidence for further patch refinement.
- If new crashes appear that differ in type or location from the original vulnerability, this may indicate that the patch has introduced regressions or new weaknesses. Such crashes are reported separately and need to be analysed.

The outcome of patch validation is thus not only a binary "pass/fail" judgment but a structured assessment of patch completeness, supported by an automatically generated suite of test inputs and detailed coverage information. This provides developers and security engineers with actionable evidence about how thoroughly a vulnerability has been addressed and where remaining risks may lie.

History

Version	Date	Status
V1.1.1	August 2026	Publication